Snarkifying Mithril Discussion

Wuyunsiqin Bingsheng Zhang
Zhejiang University, CHN
22521299@zju.edu.cn bingsheng@zju.edu.cn

August 28, 2025

1 Mithril Review

1.1 MSP-PoP

Mithril use a variant of MSP-PoP, a multisignature based on Boneh–Lynn–Shacham (BLS) signatures with proofs of possession (PoPs). Let q be a prime, G_1, G_2, G_T be cyclic groups of order q with generators $g_1 \in G_1$, $g_2 \in G_2$, and bilinear pairing $e : G_1 \times G_2 \to G_T$. Let $H_{G_1} : \{0,1\}^* \to G_1$ be a hash-to-G_1 function, $H_\lambda : \{0,1\}^* \to Z_{2^\lambda}$ a short-exponent hash, and $H_p : \{0,1\}^* \to F_p$ a collision-resistant hash.

- **MSP.Gen**(Param): Sample $sk \leftarrow Z_q$; set $mvk \leftarrow g_2^{sk}$. Compute $\kappa_1 \leftarrow H_{G_1}(\text{“PoP”} \parallel mvk)^{sk}$, $\kappa_2 \leftarrow g_1^{sk}$. Return secret key sk, verification key mvk, and proof of possession $\kappa = (\kappa_1, \kappa_2)$.
- **MSP.Check**(mvk, κ): If $e(\kappa_1, g_2) = e\big(H_{G_1}(\text{“PoP”} \parallel mvk), mvk\big)$ and $e(g_1, mvk) = e(\kappa_2, g_2)$ both hold, return 1; otherwise return 0.
- **MSP.Sig**(sk, msg): Return $\sigma \leftarrow H_{G_1}(\text{“M”} \parallel \mathsf{msg})^{sk}$.
- **MSP.Ver**($\mathsf{msg}, mvk, \sigma$): Return 1 if $e(\sigma, g_2) = e\big(H_{G_1}(\text{“M”} \parallel \mathsf{msg}), mvk\big)$, and 0 otherwise.
- **MSP.AKey**($\mathbf{mvk}$): Takes a vector $\mathbf{mvk} = (mvk_1, \ldots, mvk_d)$ of (previously checked) verification keys and returns an intermediate aggregate public key $ivk \leftarrow \prod_{i=1}^{d} mvk_i$.
- **MSP.ASig**($\boldsymbol{\sigma}$): Takes as input a vector $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_d)$ and returns $\mu \leftarrow \prod_{i=1}^{d} \sigma_i$.
- **MSP.BKey**($\mathbf{mvk}, e_{\boldsymbol{\sigma}}$): Takes a vector $\mathbf{mvk}$ of (previously checked) verification keys and a weighting seed $e_{\boldsymbol{\sigma}}$, derives $e_i \leftarrow H_\lambda(i, e_{\boldsymbol{\sigma}})$, and returns $ivk \leftarrow \prod_{i=1}^{d} mvk_i^{e_i}$.
- **MSP.BSig**($\boldsymbol{\sigma}$): Takes as input a vector of signatures $\boldsymbol{\sigma}$ and returns $(\mu, e_{\boldsymbol{\sigma}})$ where $e_{\boldsymbol{\sigma}} \leftarrow H_p(\boldsymbol{\sigma})$, $e_i \leftarrow H_\lambda(i, e_{\boldsymbol{\sigma}})$, $\mu \leftarrow \prod_{i=1}^{d} \sigma_i^{e_i}$.

1.2 Mithril protocol in Operation phase

Protocol Π.STM. Operation phase

- EligibilityCheck: On input $(\mathsf{mesg}, \mathsf{index})$, user P_i runs: Let $\overline{\mathsf{topic}} \leftarrow$ "0"$||\mathsf{AVK}||\mathsf{topic}$, $\sigma \leftarrow \mathsf{MSP.Sig}(msk, \overline{\mathsf{topic}})$; $ev \leftarrow \mathsf{MSP.Eval}(\overline{\mathsf{topic}}, \mathsf{index}, \sigma)$. Return 1 if $ev < \phi(\mathsf{stake})$, else return 0.
- CreateSig: On input $(\mathsf{mesg}, \mathsf{index})$: If $\mathsf{EligibilityCheck}(\mathsf{mesg}, \mathsf{index})$ is 1, then let $\overline{\mathsf{topic}} \leftarrow$ "0"$||\mathsf{AVK}||\mathsf{topic}$; $\sigma \leftarrow \mathsf{MSP.Sig}(msk, \overline{\mathsf{topic}})$; $\sigma_{\mathsf{body}} \leftarrow \mathsf{MSP.Sig}(msk, \text{"1"}||\overline{\mathsf{topic}}||\mathsf{body})$ and produce an individual signature $\pi = (\sigma, \sigma_{\mathsf{body}}, reg_i, i, \boldsymbol{p}_i)$, where $\boldsymbol{p}_i$ is the user's path inside the Merkle tree AVK and reg_i is $(mvk_i, \mathsf{stake}_i)$.
- Verify: On input a party P_i, a signature π, index index, and message (mesg), parse $\pi = (\sigma, \sigma_{\mathsf{body}}, reg_i, i, \boldsymbol{p}_i)$. Parse reg_i as $(mvk_i, \mathsf{stake}_i)$. Check that reg_i corresponds to party P_i, let $\overline{\mathsf{topic}} \leftarrow$ "0"$||\mathsf{AVK}||\mathsf{topic}$; $ev \leftarrow \mathsf{MSP.Eval}(\overline{\mathsf{topic}}, \mathsf{index}, \sigma)$ check that $ev < \phi(\mathsf{stake}_i)$ and check $\mathsf{MT.Check}(\mathsf{AVK}, N, (vk_i, \mathsf{stake}_i), i, \boldsymbol{p}_i) = 1$. If parsing or checking fails, return 0. Otherwise, return $\mathsf{MSP.Ver}(\overline{\mathsf{topic}}, mvk_i, \sigma) \wedge \mathsf{MSP.Ver}(\text{"1"}||\overline{\mathsf{topic}}||\mathsf{body}, mvk_i, \sigma_{\mathsf{body}})$.
- Aggregate: On input vectors $\boldsymbol{P}, \boldsymbol{\pi}, \mathbf{index}$ and message (mesg), parse $\boldsymbol{P}, \boldsymbol{\pi}$ and $\mathbf{index}$ as a vector $P_j, \pi_j, \mathsf{index}_j$ of size k, let $\overline{\mathsf{topic}} \leftarrow$ "0"$||\mathsf{AVK}||\mathsf{topic}$ and run $\mathsf{Verify}(P_j, \mathsf{index}_j, \mathsf{mesg}, \pi_j)$.
  If parsing or checking fails, return $\perp$. If any $\mathsf{index}_j = \mathsf{index}_i$ for $j \neq i$ return 0. Otherwise, parse $\pi_j = (\sigma_j, \sigma_{\mathsf{body},j}, reg_j, i_j, \boldsymbol{p}_j)$ and reg_j as $(mvk_j, \mathsf{stake}_j)$. Let $ivk \leftarrow \mathsf{MSP.BKey}(\mathbf{mvk}, \boldsymbol{\sigma})$,$ivk_{\mathsf{body}} \leftarrow \mathsf{MSP.AKey}(\mathbf{mvk})$, $\mu \leftarrow \mathsf{MSP.BSig}(\boldsymbol{\sigma})$, $\mu_{\mathsf{body}} \leftarrow \mathsf{MSP.ASig}(\boldsymbol{\sigma}_{\mathsf{body}})$, set $x = (\mathsf{AVK}, ivk, \mu, e_{\boldsymbol{\sigma}}, \mathsf{mesg})$ and $\boldsymbol{w} = (mvk_j, \mathsf{stake}_j, \boldsymbol{p}_j, ev_j, \sigma_j, \mathsf{index}_j)$ for $j \in \{1 \ldots k\}$. Then, $\pi_{avk} \leftarrow \mathsf{PS.P}(\mathsf{PS.RS}, x, \boldsymbol{w})$. Return $\tau = (ivk, \mu, e_{\boldsymbol{\sigma}}, ivk_{\mathsf{body}}, \mu_{\mathsf{body}}, \pi_{avk})$.
- VerifyAggregate: On input (τ, mesg), parse $\tau \rightarrow (ivk, \mu, e_{\boldsymbol{\sigma}}, ivk_{\mathsf{body}}, \mu_{\mathsf{body}}, \pi_{avk})$, check that $\mathsf{PS.V}(\mathsf{PS.RS}, (\mathsf{AVK}, ivk, \mu, e_{\boldsymbol{\sigma}}, \mathsf{mesg}), \pi_{avk})$ is true. If parsing and checking is successful, let $\overline{\mathsf{topic}} \leftarrow$ "0"$||\mathsf{AVK}||\mathsf{topic}$ and return $\mathsf{MSP.Ver}(\overline{\mathsf{topic}}, ivk, \mu) \wedge \mathsf{MSP.Ver}(\text{"1"}||\overline{\mathsf{topic}}||\mathsf{body}, ivk_{\mathsf{body}}, \mu_{\mathsf{body}})$.

1.3 Relation for Mithril

The proof system is defined over the language $\mathcal{L}_{\mathsf{avk}}$, which describes statements x that admit a witness w such that $R_{\mathsf{avk}}(x, w) = 1$. Concretely, statements and witnesses are

$$x = (\mathsf{AVK},\, ivk,\, ivk_{\mathsf{body}},\, \mu,\, e_{\boldsymbol{\sigma}},\, \mathsf{mesg}), \qquad w = \left\{(mvk_i,\, \mathsf{stake}_i,\, p_i,\, ev_i,\, \sigma_i,\, \mathsf{index}_i)\right\}_{i=1}^{k}.$$

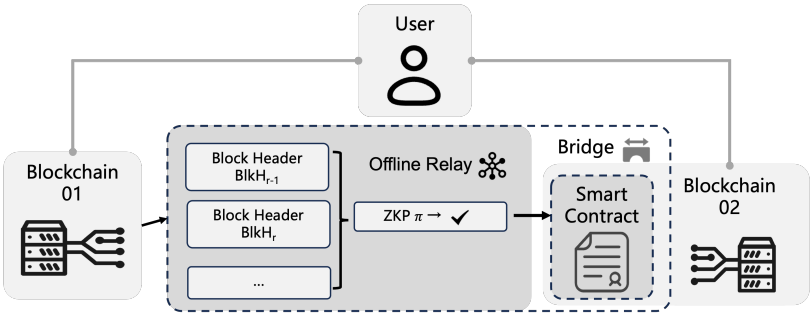
The relation R_{avk} is parameterized by public values (N, m, k, ϕ).

$R_{\mathsf{avk}}(x, w) = 1$ if and only if all of the following hold:

- $ivk = \mathsf{MSP.BKey}(\mathbf{mvk},\, e_{\boldsymbol{\sigma}})$ and $ivk_{\mathsf{body}} = \mathsf{MSP.AKey}(\mathbf{mvk})$, where $\mathbf{mvk} = (mvk_1, \ldots, mvk_k)$.
- $(\mu,\, e_{\boldsymbol{\sigma}}) = \mathsf{MSP.BSig}(\boldsymbol{\sigma})$, where $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_k)$.
- For all i: $\mathsf{index}_i \leq m$, and for all $i \neq j$: $\mathsf{index}_i \neq \mathsf{index}_j$.

- For each $i \in \{1, \ldots, k\}$: the pair $(mvk_i, \mathsf{stake}_i)$ lies in the Merkle tree with root AVK and universe size N, following path p_i.
- For each $i \in \{1, \ldots, k\}$: $\mathsf{MSP.Eval}(\mathsf{topic}, \mathsf{index}_i, \sigma_i) = ev_i$.
- For each $i \in \{1, \ldots, k\}$: $ev_i \leq \phi(\mathsf{stake}_i)$.

1.4 Why Snarkify Mithril —— ZK Cross-Chain Bridge



We SNARKify Mithril to build practical cross-chain bridges: the aggregator proves off-chain that a Mithril certificate is valid (weighted BLS aggregations, eligibility, index and Merkle checks), and the destination chain verifies only a succinct proof. This shifts heavy pairing/MSM/hash work off-chain, makes on-chain cost small and stable (single verifier call), removes the need for native BLS support, and preserves Mithril's stake-weighted security guarantees under an untrusted relay.

2 Snarkifying Mithril (Practical)

This section implements a Halo2-based zero-knowledge cross-chain protocol instantiated with Mithril. The circuit covers: (i) unweighted BLS multisignature aggregation ($\mathsf{MSP.A}$), (ii) seed-weighted BLS aggregation ($\mathsf{MSP.B}$), (iii) lottery index range & uniqueness checks, (iv) eligibility/threshold checks against stake, and (v) Merkle membership checks for registered signers. The experimental environment matches the previous section.

2.1 Experimental Tasks

- **MSP.A (baseline aggregation).** Implements the straightforward BLS aggregation and in-circuit pairing check $e(\mu, g_2) = e(H_{G_1}(\text{“M”} \| msg), ivk)$. We measure proving time, proof size, and verification time as the number of aggregated signatures grows.
- **MSP.B (seed-weighted aggregation).** Extends aggregation with per-signer short exponents $e_i \leftarrow H_\lambda(i, e_{\boldsymbol{\sigma}})$ and $(\mu, e_{\boldsymbol{\sigma}})$ where $e_{\boldsymbol{\sigma}} \leftarrow H_p(\boldsymbol{\sigma})$. The

circuit proves the weighted pairing equation. We observe higher proving cost due to hashing and multi-scalar multiplications (MSMs) with weights.

- **Lottery index check.** Ensures $\mathsf{index}_i \leq m$ for all i and $\mathsf{index}_i \neq \mathsf{index}_j$ for $i \neq j$. The uniqueness constraint induces $O(n^2)$ comparisons; we report scaling behavior.
- **Eligibility/threshold check.** For each signer, enforces $ev_i = \mathsf{MSP.Eval}(\mathsf{topic}, \mathsf{index}_i, \sigma_i)$ and $ev_i \leq \phi(\mathsf{stake}_i)$. The circuit cost grows roughly linearly in the number of checked signers.

2.2 Benchmark Tables

Table 1: MSP.A (unweighted aggregation) results

aggregated signatures	**2**	**200**	**2000**	**4000**	**6000**	**8000**
Proving time (s)	19.17	29.51	72.89	123.67	181.38	226.84
Proof size (bytes)	3072	4128	11424	18944	26944	34944
Verification time (ms)	20.93	9.09	19.95	16.03	20.25	55.16

Table 2: MSP.B (seed-weighted aggregation) results

aggregated signatures	**8**	**16**	**32**	**64**	**128**	**256**
Proving time (s)	35.73	48.48	65.37	103.75	184.02	327.30
Proof size (bytes)	5856	7936	10944	17312	30240	55744
Verification time (ms)	10.30	14.68	15.05	14.54	24.51	69.35

Table 3: Lottery index check results (range & uniqueness)

indices	**128**	**256**	**512**	**1024**	**2048**
Proving time (s)	4.08	6.77	13.41	37.39	122.49
Proof size (bytes)	960	1920	4576	14944	14944
Verification time (ms)	5.05	6.15	9.42	15.10	13.08

Table 4: Eligibility / stake-threshold check results

checks	**128**	**256**	**512**	**1024**	**2048**
Proving time (s)	10.14	12.78	17.36	26.39	44.56
Proof size (bytes)	1344	1920	2848	4800	8832
Verification time (ms)	6.77	6.39	7.10	11.24	12.59

Table 5: Merkle membership proof results (transposed, with stake)

	16	**32**	**64**	**128**	**256**	**512**	**1024**
original proofs	32	64	128	256	512	1024	2048
Proving time (s)	5.68	7.60	13.44	23.29	48.63	100.05	213.25
Proof size (bytes)	704	1056	2112	3872	8448	17600	38016
Verification time (ms)	6.57	4.47	7.64	8.13	11.65	13.61	20.31

3 Snarkifying Mithril(Theoretical)

3.1 SNARK

A SNARK for a relation $\mathcal{R}$ is a triple of PPT algorithms $\Pi = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ defined as follows:

- $\mathsf{srs} \leftarrow \mathsf{Setup}(1^\lambda)$:On input security parameter λ , it outputs a structured reference string srs.
- $\pi \leftarrow \mathsf{Prove}(\mathsf{srs}, x, w)$: On input srs, a statement x and a witness w such that $(x, w) \in \mathcal{R}$, output a proof π.
- $0/1 \leftarrow \mathsf{Verify}(\mathsf{srs}, x, \pi)$:On input srs, a statement x, and a proof π, it outputs either 1 indicating accepting the statement or 0 for rejecting it.

3.2 BLS

$\Pi_{BLS} = (Setup, Sign, Ver)$

- $(sk, vk = g_1^{sk}) \leftarrow Setup()$
- $\sigma \leftarrow Sign(sk, msg)$, where $\sigma = H(msg)^{sk}$
- $1/0 \leftarrow Ver(vk, \sigma, msg)$, check $e(g_1, \sigma) = e(\mathsf{vk}, H(msg))$

3.3 Merkle tree

$\Pi_{MT} = (Create, Check)$

- $avk \leftarrow Create(\mathbf{VK})$
- $1/0 \leftarrow Check(vk, avk, path)$

3.4 SNARK-based Mithril

$\Pi_{\mathsf{F-Mithril}} = (\mathsf{Register}, \mathsf{EligibilityCheck}, \mathsf{CreateSig}, \mathsf{Verify}, \mathsf{Aggregate}, \mathsf{VerifyAggregate})$

$\Pi_{\mathsf{SNARK-Mithril}} = (\mathsf{PGen}, \mathsf{KGen}, \mathsf{Avk}, \mathsf{AvkProve}, \mathsf{AvkCheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$

Mithril.PGen()

$$srs \leftarrow \mathrm{Setup}(1^\lambda)$$
$$\text{return } srs$$

Mithril.KGen(pp)

$$sk \leftarrow Z_q$$
$$vk \leftarrow g_1^{sk}$$
$$\text{return } (sk, vk)$$

Mithril.Avk(VK, w)

$$\mathbf{WVK} \leftarrow \{(w_i, vk_i) : i \in [n]\}$$
$$w \leftarrow \sum_{i \in [n]} w_i$$
$$avk \leftarrow MT.\mathrm{Create}(\mathbf{WVK})$$
$$\text{return } (avk, w)$$

Mithril.AvkProve(VK, w, avk, path srs)

$$\pi_1 \leftarrow \mathsf{SNARK.Prove}(srs, x = avk, w = (\mathbf{VK}, \mathbf{w}, \mathbf{path}))$$
$$(\mathcal{R} = \{(x = avk, w = (\mathbf{VK}, \mathbf{w}, \mathbf{path})) \quad | \quad \forall i \in [m],\ MT.\mathrm{Check}(avk, (w_i, vk_i), path_i) = 1\})$$
$$\text{return } \pi_1$$

Mithril.AvkCheck(avk, srs)

$$\text{return } \mathsf{SNARK.Verify}(srs, x = avk, \pi_1)$$

Mithril.Sig(sk, j, msg, w, $\mathbf{w}_i$)

$$\sigma \leftarrow \mathrm{BLS.Sign}(sk, msg)$$
$$\text{return } \sigma$$

Mithril.Ver(vk, j, msg, σ, w, w_i)

$$\text{return } \mathrm{BLS.Ver}(vk, \sigma, msg)$$

Mithril.ASig(S=$\{(\sigma_j, j) | j \in [m]\}$**)**

$$\boldsymbol{\sigma} \leftarrow [\{\sigma_j | j \in [m]\}]$$
$$e_\sigma \leftarrow H_p(\boldsymbol{\sigma})$$
$$e_j \leftarrow H_\lambda(j, e_\sigma)$$
$$\mu \leftarrow \prod_j \sigma_j^{e_j}$$
$$\pi_2 \leftarrow SNARK.Prove(srs, x = (\mu, e_\sigma), w = (\boldsymbol{\sigma}))$$
$$(R = \{x = (\mu, e_\sigma, , w), w = (\boldsymbol{\sigma}) \quad |$$
$$e_\sigma = H_p(\boldsymbol{\sigma}) \wedge e_j = H_\lambda(j, e_\sigma) \wedge \mu = \prod_j \sigma_j^{e_j}\})$$
$$\sigma^* \leftarrow (\mu, e_\sigma, \pi_2)$$
return σ^*

Mithril.AVer(ivk, msg, $\sigma*, w$)

$$r_1 \leftarrow \mathsf{SNARK.Verify}(srs, x = (\mu, e_\sigma), \pi_2)$$
$$r_2 \leftarrow \mathsf{BLS.Ver}(ivk, \mu, msg)$$
return $r_1 \wedge r_2$

(Here, ivk $= \prod_j vk_j^{e_j}$)

3.5 Remained Questions

1. It is unclear how to incorporate weights into BLS signatures, especially when performing aggregation in BLS multisignatures.

   In the case where all signers agree on the same message m, the aggregated signature can be written as

   $$\sigma_{agg} = \prod_i \sigma_i = H(m)^{\sum_i sk_i},$$

   and thus the corresponding aggregated public key is simply

   $$vk_{agg} = \prod_i vk_i.$$

   Verification then reduces to a single pairing equation

   $$e(g, \sigma_{agg}) \stackrel{?}{=} e(vk_{agg}, H(m)),$$

   which is compact and efficient.

   However, when each signer signs a *different* message m_i, the aggregated signature becomes

   $$\sigma_{agg} = \prod_i H(m_i)^{sk_i},$$

for which there does not exist a single aggregated public key. Instead, the verification requires

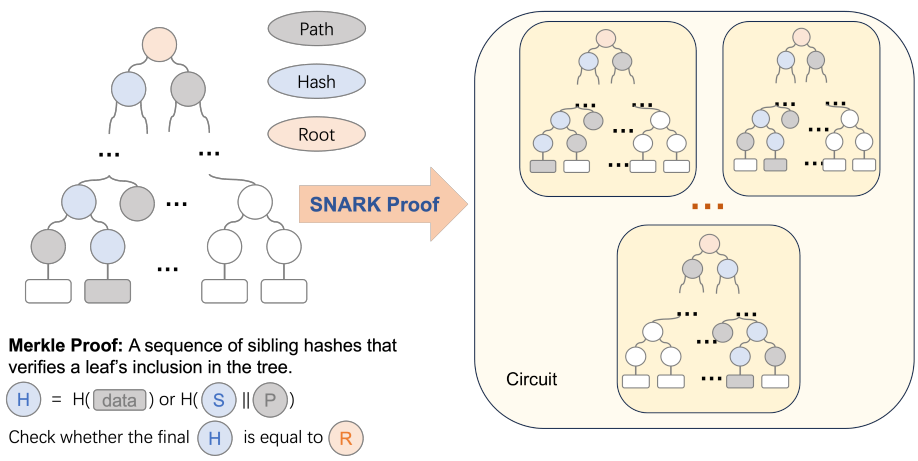
$$e(g, \sigma_{agg}) \stackrel{?}{=} \prod_i e(vk_i, H(m_i)),$$

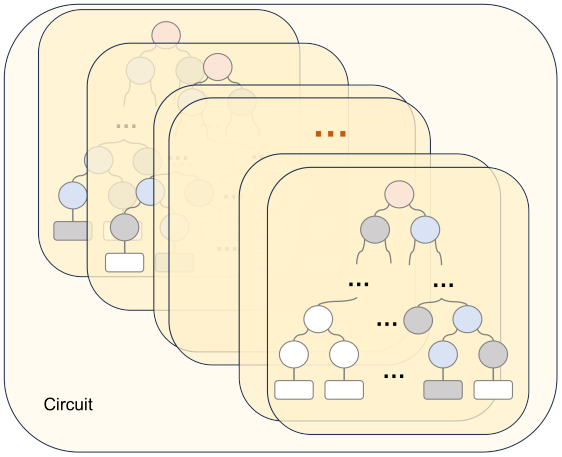
where every individual public key vk_i and message m_i must be explicitly included in the verification. This implies that, unlike the single-message case, the verification cost is not succinct, and the aggregation does not yield a compact verification procedure.

2. The smart contract needs to verify both the Mithril.AvkCheck and Mithril.AVer proofs, i.e., it must verify two SNARK proofs and one BLS signature.

4 Other Explorations

We designed an optimization for workloads that require opening a large number of Merkle proofs. Due to scenario-specific constraints, this approach may not be directly applicable to our current use case.

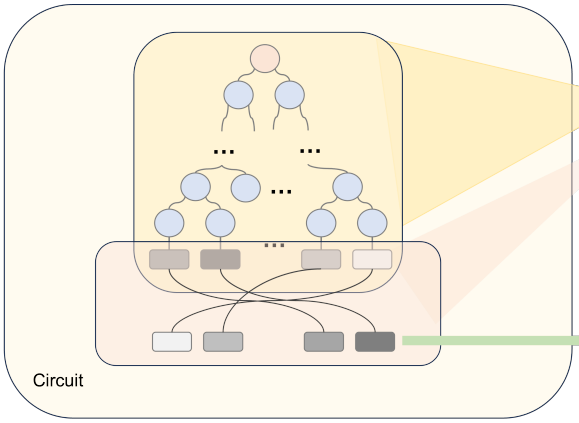




Relation:

$$\mathcal{R} = \{(x = (C_{root}, y, f), w = (\mathbf{M}, \mathbf{I}, \mathbf{P_I})) \mid \forall i \in \mathbf{I},\ \text{Merkle.Verify}(m_i, P_i, C_{root}) = 1 \ \wedge \ y = f(\mathbf{M}, \mathbf{I})\},$$

Heavy Work!



Relation:

$$\mathcal{R} = \{(x = (C_{root}, y, f), w = (\mathbf{M}, \mathbf{I}, \mathbf{I}', \sigma)) \mid \text{Merkle.Construct}(\mathbf{M}) = C_{root} \ \wedge \ \vec{v}_{[n]} = \sigma(\mathbf{I} || \mathbf{I}') \ \wedge \ y = f(\mathbf{M}, \mathbf{I})\}.$$

More Efficient when the number of opening nodes is more than n/log(n).

How to encode elliptic curve point ?

Elliptic Curve point → **Encoding** → (x , lsb(y))

To prove $\mathrm{M} = \mathrm{M_I} \cup \mathrm{M_{I'}}$ → Permutation Argument

$$h = \prod_{\forall (x,y) \in \mathrm{M_I}} (x + \gamma \cdot lsb(y) + \zeta)$$

$$\mathrm{g} = \prod_{\forall (x,y) \in \mathrm{M_{I'}}} (x + \gamma \cdot lsb(y) + \zeta)$$

$$\mathrm{m} = \prod_{\forall (x,y) \in \mathrm{M}} (x + \gamma \cdot lsb(y) + \zeta)$$

To verify the permutation, we check the equality:

$$h \cdot g = m.$$